

AI Event Ticket Booking Agent — Project Proposal

Authors: Nawaf Almufarej, Dana, Hessa, Refal

Program: Agentic AI Bootcamp — Capstone

Date: 2026-05-20

Status: Proposal — not yet implemented

1. Business Problem

Domain

Event ticketing and live-entertainment commerce — concerts, sports, theatre, conferences, and cultural events. End-to-end: discovery, seat selection, member pricing, payment, and ticket fulfilment.

Pain Points

The current ticket-buying experience is full of friction. Across global and Saudi platforms, users routinely face:

- **Slow, multi-page checkout** — find event → pick category → pick seat → register → pay → confirm, with each step a new screen.
- **Confusing seat selection on mobile** — pinch-to-zoom venue maps that are unreadable on small screens.
- **Member discounts buried or forgotten** — members are entitled to discounts they don't apply because the UI doesn't surface them.
- **Multi-ticket coordination is painful** — buying 4 tickets seated together is rarely a smooth flow.
- **Payment friction** — too many redirects, Mada/Apple Pay/STC Pay support varies, sessions time out mid-checkout.
- **Cart abandonment** — industry reports put ticketing cart-abandonment rates among the highest in e-commerce.
- **Poor receipts and ticket delivery** — emails get lost in spam, PDF tickets are hard to find later, QR codes fail to scan.
- **Discoverability gap** — finding "what's happening this weekend" is harder than it should be.
- **Customer-support load** — venues and organisers get DMed on social media for things the platform should handle.

Concrete Example — WeBook

Take **WeBook**, Saudi's national ticketing platform, as the running example. WeBook hosts an enormous live catalog at any given moment — Riyadh Season events, MDLBEAST Soundstorm, Saudi Pro League matches, Formula 1, theatre, family shows, cultural events — all in one place. The catalog density is great for organisers but a problem for buyers:

- A new user lands on the homepage with **no clear starting point**; search and category filters exist but they don't *learn* the user.

- A user who books football matches and rock concerts never gets recommendations rooted in their **actual behaviour** — only generic "popular now" carousels and editorial banners.
- When a user is clearly **not happy** with the suggested results ("not what I'm looking for", repeated rejections, vague queries), the platform has no way to detect that and re-frame the conversation — it serves the same filters and the user gives up.
- The result is a busy, well-stocked catalog that **converts worse than it should** because the experience is generic and doesn't read the buyer.

This is the gap a chat agent can close: **remember preferences across sessions, suggest events rooted in real behaviour, and detect frustration in real time** — adapting the conversation instead of forcing the user to fight the catalog.

Business Impact

- Lost revenue from abandoned carts and unsold seats.
- Members who never realise they had a discount → reduced loyalty value.
- High support cost from manual ticket reissues, refund queries, "I never got my email" complaints.
- Poor data on which events / categories convert best.

2. Market Research

2.1 Global Event-Ticketing Market

The global event-ticketing market is large and structurally growing. Publicly cited analyst estimates put the market at roughly **USD 55–70 billion in 2024**, with a **CAGR of ~5–7%** through 2030, and the AI-in-ticketing sub-segment growing materially faster (Grand View Research; Fortune Business Insights; Mordor Intelligence; Statista — see §2.5).

Leading players (global).

Vendor	Position	Notes
Ticketmaster (Live Nation)	#1 globally	Dominant in North America / EU; high-value enterprise contracts
Eventbrite	SMB / mid-market	Self-serve event creation; popular with smaller organisers
AXS	Sports + concerts	Vertically integrated with venues
SeatGeek	Aggregator + primary	Strong UX, "Deal Score" ranking
StubHub / viagogo	Secondary market	Resale-led
DICE	Mobile-first / curated	No-fees positioning, AI-curated discovery

Key global trends.

- **AI moving from add-on to core** — dynamic pricing, demand forecasting, fraud detection, conversational chatbots, recommendation engines.
- **Mobile-first / single-page checkouts** — every major platform is collapsing the funnel.

- **Member / fan-loyalty programmes** as a differentiator (Ticketmaster Verified Fan, AXS Fan Account).
- **Cart abandonment is the #1 unsolved UX problem** — widely reported as 60–80% in ticketing, materially higher than typical e-commerce.
- **Secondary-market regulation** is tightening globally — primary platforms are pulled deeper into identity/verification.

2.2 Local Market — Saudi Arabia

Macro drivers.

- **Vision 2030** has explicit quality-of-life and entertainment-sector targets. The **General Entertainment Authority (GEA)** was created in 2016 specifically to grow the live-entertainment industry.
- **Riyadh Season** has become one of the largest entertainment festivals in the world; **MDLBEAST Soundstorm** is among the largest music festivals in the region; **Formula 1 Saudi Arabian GP**, **Saudi Pro League**, **LIV Golf**, and **Esports World Cup** all run on Saudi soil and sell tickets in volume.
- **Cloud and SaaS adoption** in the Kingdom is rapid (CITC, SDAIA), and **digital payments** are near-universal (Mada, Apple Pay, STC Pay).
- **Saudi VAT** of 15% applies to ticketed events; **ZATCA** governs invoicing and tax reporting.

Saudi ticketing landscape.

- **WeBook.com** — Saudi's main national ticketing platform, used for GEA events and Riyadh Season.
- **Platinumlist** — regional player active in Saudi and the wider Gulf.
- Numerous event-specific or venue-specific platforms exist; the space is **fragmented below the WeBook tier**.
- **WhatsApp** is the dominant business-comms channel, but ticket platforms treat it as an afterthought.

Saudi-specific pain points (beyond the global ones).

- **Bilingual reality.** Saudi buyers routinely mix Arabic and English in the same message — ticket UIs rarely handle code-switching.
- **Membership programmes underused.** Many Saudi cultural and sports memberships exist (clubs, GEA-tied programmes, university alumni) but discount flows are clunky and unevenly applied at checkout.
- **Mada / Apple Pay / STC Pay** are mandatory expectations, not "nice to have" — anything that doesn't support them feels foreign.
- **PDPL alignment.** Saudi's **Personal Data Protection Law (PDPL)** is in force; storing names, emails, and ticket data has explicit compliance obligations.

2.3 Competitive Positioning

Vector	Ticketmaster	WeBook	Eventbrite	Our Agent
Conversational booking	Limited chatbot	None	None	Core, end-to-end

Vector	Ticketmaster	WeBook	Eventbrite	Our Agent
Bilingual Arabic/English NLU	Weak	Partial UI	Weak	Designed-in
Member-discount UX	Hidden behind login	Manual	Manual	Auto-detected at quote
Multi-ticket cap by tier	Static rules	Static	Static	Membership-aware cap
Seat-map in chat	No	Web only	Web only	Generated inline image
WhatsApp-native flow	No	No	No	Stretch goal, prioritised
Setup complexity for organisers	High	Medium	Low	Low (chat-first)
Preference learning across sessions	Limited	None	None	Core, per-user profile
Frustration / sentiment detection	None	None	None	Built-in, adapts the flow

We are not competing with Ticketmaster on enterprise breadth. We are competing on **booking friction**: how few seconds it takes a Saudi user to go from "I want a ticket" to "ticket in my inbox" — in their own language, with their member discount auto-applied, paid via Mada.

2.4 Market Gap & Opportunity

The opening for this project sits at the intersection of six under-served gaps:

- 1 **The checkout-friction gap.** Cart abandonment is the #1 unsolved UX problem in ticketing. A conversational agent that completes the booking in a single chat thread attacks the root cause.
- 2 **The bilingual gap.** No major Saudi or global ticketing platform handles Arabic-English code-switched natural language as a first-class input.
- 3 **The member-experience gap.** Memberships exist everywhere, but the *application* of member benefits at the moment of purchase is consistently bad. An agent that detects membership by email lookup and surfaces the discount *before* the user asks fixes a real revenue-leak.
- 4 **The discoverability gap.** "What's happening this weekend in Riyadh?" is a query no current platform answers conversationally. A chat-first agent with a multi-event catalog (concerts, sports, conferences) and rich event cards in-line addresses this directly.
- 5 **The personalisation gap.** WeBook, Platinumlist, and the global incumbents all serve generic "popular now" recommendations. None adapt to *this* user based on their booking history, rejected categories, and stated preferences. A chat agent that builds a per-user profile from the first interaction and weights suggestions accordingly is a meaningful step-change over the status quo.
- 6 **The behaviour-awareness gap.** No current ticketing platform detects when a user is frustrated or unhappy with the suggestions and re-frames the conversation. The agent will classify sentiment on every turn (engaged / neutral / frustrated) and switch strategy when frustration is detected.

Secondary opportunities.

- **WhatsApp-native ticketing** — forwarding a poster to the agent and booking in-thread is something no incumbent does cleanly.
- **HITL confirmation** as a trust feature, not just a graded requirement — aligns with PDPL and Saudi user expectations.

- **Voice booking** for hands-busy contexts (driving, accessibility).

Target beachhead. Saudi cultural, sports, and conference events sold by small-to-mid organisers — sectors where the team has access for design partners and demo feedback, and where Riyadh Season / GEA programming creates a constant flow of real-world test scenarios.

2.5 References

Figures and claims in this section are drawn from publicly available reports and government sources. Specific numbers should be re-verified against the latest published reports at submission time.

Global event-ticketing market sizing and trends.

- Grand View Research — *Event Ticketing Market Size & Share Report* — <https://www.grandviewresearch.com/industry-analysis/event-tickets-market>
- Fortune Business Insights — *Online Event Ticketing Market* — <https://www.fortunebusinessinsights.com/>
- Mordor Intelligence — *Online Event Ticketing Market* — <https://www.mordorintelligence.com/industry-reports/online-event-ticketing-market>
- Statista — *Event Tickets – Worldwide* — <https://www.statista.com/outlook/dmo/eservices/event-tickets/worldwide>
- IDC — vertical market research — <https://www.idc.com/>

Vendor positioning.

- Ticketmaster — <https://www.ticketmaster.com/>
- Live Nation Entertainment — <https://www.livenationentertainment.com/>
- Eventbrite — <https://www.eventbrite.com/>
- AXS — <https://www.axs.com/>
- SeatGeek — <https://seatgeek.com/>
- StubHub — <https://www.stubhub.com/>
- DICE — <https://dice.fm/>

Saudi Arabia — entertainment, ticketing, payments, regulation.

- Saudi Vision 2030 — official program site — <https://www.vision2030.gov.sa/>
- General Entertainment Authority (GEA) — <https://gea.gov.sa/en>
- Riyadh Season — <https://riyadhseason.sa/>
- MDLBEAST / Soundstorm — <https://mdlbeast.com/>
- Formula 1 Saudi Arabian Grand Prix — <https://saudiarabiangp.com/>
- WeBook (Saudi national ticketing) — <https://webook.com/>
- Platinumlist — <https://platinumlist.net/>
- CITC — Communications, Space & Technology Commission — <https://www.citc.gov.sa/en>
- SDAIA — Saudi Data and AI Authority — <https://sdaia.gov.sa/en>
- MCIT — Ministry of Communications and Information Technology — <https://www.mcit.gov.sa/en>

- ZATCA — Zakat, Tax and Customs Authority (VAT, e-invoicing) — <https://zatca.gov.sa/en>
- SAMA — Saudi Central Bank (payments) — <https://www.sama.gov.sa/en-US>
- SDAIA PDPL portal — <https://sdaia.gov.sa/en/SDAIA/about/Pages/PersonalDataProtection.aspx>

Payment gateway.

- Moyasar — Saudi payments gateway — <https://moyasar.com/>
-

3. Potential AI Solutions

Several AI-driven approaches could address these pain points:

- 1 **Conversational booking agent (chosen approach).** A chat-first agent that takes the user from intent ("I want a ticket") to ticket delivery in one thread. Handles event search, disambiguation, category/seat selection, member-discount application, seat-hold timer, payment redirect, and ticket fulfilment.
- 2 **Preference-aware recommendation + sentiment-aware concierge (built-in).** Natural-language discovery — "what concerts this weekend?", "any family-friendly events in Riyadh?" — but weighted by a per-user preference profile (past bookings, rejected categories, language). Combined with real-time sentiment classification on every turn so the agent notices frustration and re-frames instead of repeating itself.
- 3 **Voice-driven booking (stretch).** Speech-to-text feeding the same agent — useful for accessibility and hands-busy contexts.
- 4 **WhatsApp-native channel (stretch).** Same agent exposed over the WhatsApp Business API.
- 5 **Dynamic-pricing assistant (out of MVP scope).** Suggests price points for organisers based on demand signals — separate product, future work.
- 6 **Multi-agent orchestration (stretch).** Specialised agents for catalog, pricing, hold, payment, fulfilment via LangGraph.

We propose to build option **(1)** for the MVP with **(2)** baked in, and to treat **(3)(4)(6)** as stretch goals to attempt once the MVP is stable.

4. High-Level Solution Design

What we will build

A headless AI Event Ticket Booking Agent: a chat interface backed by a single agent that will take a user from "I want a ticket" to a confirmed, paid, emailed ticket — without leaving the conversation. Rich elements (event cards with image + URL, seat-map images, payment redirect, ticket card) are rendered in-line in the chat.

Core Loop

Listen → Identify event → Quote (with member discount) → Select seats → Hold (timer) → Confirm → Pay → Deliver ticket.

Planned MVP Features

#	Feature	Description
F1	Conversational intent + event search	User says what they want; agent searches the events catalog.
F2	Event disambiguation with rich cards	When matches are ambiguous, agent posts inline event cards (image + URL + date + venue).
F3	Email-based membership lookup	Agent asks for email, looks up the <code>members</code> table, applies tier.
F4	Multi-ticket quantity prompt	Agent asks "how many tickets?" — required before pricing.
F5	Membership-aware ticket cap	Normal users capped at N tickets; members capped at M ($M > N$). Agent enforces and explains.
F6	Category & price quote	Shows category prices, member discount applied inline, VAT 15% line item, total.
F7	Visual seat selection	Agent renders a generated seat-map image (available / sold / held) and accepts seat IDs.
F8	Seat-hold timer (10 min)	Selected seats locked atomically; expiry timestamp echoed back to user.
F9	HITL confirmation before payment	Agent shows full booking summary and requires explicit approval before issuing payment URL. (<i>Mandatory — graded design requirement.</i>)
F10	Moyasar payment redirect	Agent generates a sandbox checkout session, posts the link.
F11	Webhook callback + idempotency	Server-to-server confirmation; signature verified; idempotency key on booking.
F12	Ticket PDF + QR + email	After payment, generate a PDF ticket with HMAC-signed QR code; email it to the buyer; surface a download link in chat.
F13	Preference-aware suggestions	Agent remembers past bookings, stated interests, and previously rejected categories per email; surfaces relevant events proactively instead of showing the catalog flat.
F14	Sentiment & behaviour awareness	Agent classifies each user turn as <code>engaged</code> / <code>neutral</code> / <code>frustrated</code> ; adapts strategy when frustration is detected (different category, more open question, alternative event type, human handoff in stretch).

Example Interaction

User: "I want a ticket for the Coldplay show."

Agent: "I found two upcoming Coldplay events. Which one?"

(renders two inline event cards: image + title + date + venue + 'Select')

User: "The Riyadh one."

Agent: "Got it — Coldplay, Kingdom Arena, 12 June 2026, 8 PM. What's your email so I can check for any member discount?"

User: "nawaf@example.com"

Agent: "You're a Platinum member — you get 15% off and can book up to 8 tickets. How many tickets do you want?"

User: "4."

Agent: (shows category card)

"Categories:

- VIP — 1,500 → **1,275** SAR each
- Gold — 800 → **680** SAR each
- Silver — 400 → **340** SAR each

Which category?"

User: "Gold."

Agent: (renders Gold-section seat-map image with available/sold/held seats labelled)

"Here's the Gold section. Pick 4 seats."

User: "G12, G13, G14, G15."

Agent: (places hold; 10:00 timer starts)

"Holding G12–G15 for 10 minutes. Summary:

- 4× Gold @ 680 SAR = 2,720 SAR
- VAT 15% = 408 SAR
- **Total: 3,128 SAR**

Confirm to proceed to payment?"

User: "Yes."

Agent: "Here's your secure payment link → [moyasar.com/checkout/sess_...]

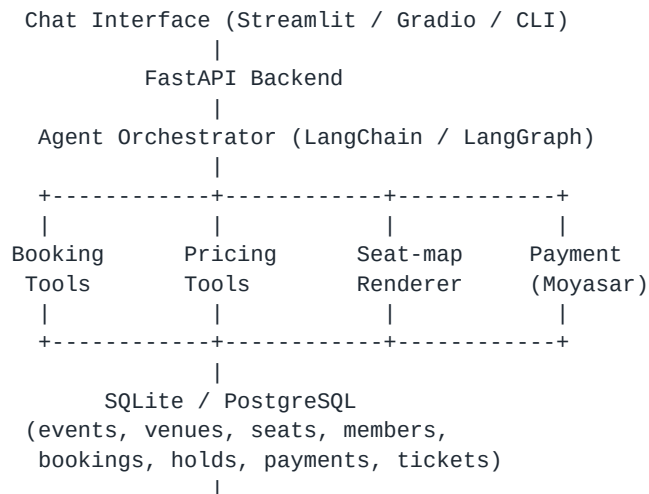
I'll be here when you're back."

(user pays; Moyasar webhook fires; server marks booking paid; ticket PDF generated)

Agent: "Payment confirmed. Here's your ticket — also sent to nawaf@example.com."

(renders ticket card: event, date, seats, QR thumbnail, **Download PDF** button)

Architecture



Tools (Python functions the agent will call)

`search_events` , `get_event_details` , `render_event_card` , `lookup_member_by_email` ,
`get_ticket_cap` , `get_categories_with_pricing` , `render_seat_map` ,
`check_seat_availability` , `place_seat_hold` , `release_seat_hold` , `compute_quote`
(base + discount + VAT), `create_booking` , `create_payment_session` (Moyasar),
`handle_payment_webhook` , `generate_ticket_pdf` , `send_ticket_email` ,
`get_booking_status` , `get_user_preferences` , `update_user_preferences` ,
`recommend_events` , `classify_sentiment` .

Memory Design

- **Short-term (session).** Current event in focus, current cart (category, quantity, selected seats), active hold (seat IDs + expiry timestamp), pending approval state, payment session ID, last error, **rolling sentiment** (last N turns classified engaged/neutral/frustrated).
- **Long-term (DB).** Events catalog, venues, seat layouts, members, bookings, payments, tickets, audit log of stage changes, **per-user preferences** (preferred categories, venues, price band, rejected categories, language), **interaction log** for sentiment review and personalisation training data.

Booking-Hold Mechanics (graded best-practice)

- Hold = atomic operation: either all requested seats are held or none are.
- TTL = 10 minutes (industry standard).
- Implementation: DB row-lock on `seats` rows + `holds` table with `expires_at` , OR Redis `SETNX` with TTL.
- Background sweeper releases expired holds.
- On payment success: hold → confirmed booking. On expiry: hold released, agent tells user.

HITL Confirmation (mandatory)

The agent **must** show the full booking summary (event, seats, base price, discount, VAT, total, hold expiry) and require explicit user approval *before* issuing the payment URL. No payment session is created without it.

Personalisation & Sentiment

Two cross-cutting capabilities make this agent feel different from a search bar wrapped in chat — they are the answer to the WeBook gap described in §1.

1. Preference memory. A `user_preferences` table keyed by email stores:

- preferred categories (concert / sports / theatre / family / cultural),
- preferred venues and cities,
- price band the user typically buys in,
- rejected categories (events they actively said no to),
- preferred language (Arabic / English / mixed).

The agent reads this on every turn and weights its event suggestions accordingly. After each booking, the profile is updated automatically. A user who has bought two football matches and a rock concert will be shown sports + music *first*, not the generic homepage carousel.

2. Sentiment & behaviour detection. The agent classifies each user turn as `engaged`, `neutral`, or `frustrated` using the same LLM that drives the booking flow. Frustration signals — "*none of these*", "*I don't like this*", repeated rejections, vague replies, abrupt single-word responses — trigger a behaviour change:

- Stop pushing the same category.
- Ask a more open clarifying question instead of narrowing further.
- Offer to browse a different event type entirely.
- Surface a member-discount or promotion to recover the conversation.
- (*Stretch*) Escalate to a human-support handoff.

The sentiment label is logged per turn into the `interaction_log` table for later review and for evaluating whether the adaptive strategy actually improves outcomes.

This is what makes the agent feel like a concierge, not a chatbot over a database.

Example — Preference + Sentiment in action

User: "I want to go out this weekend."

Agent: (reads `user_preferences` — *past bookings: 2× football, 1× rock concert; preferred city: Riyadh; sentiment: engaged*)

"Welcome back! Based on what you've booked before, here are 3 things in Riyadh this weekend you might like:"

(renders three inline event cards — *Al-Hilal match, indie-rock night, F1 practice*)

User: "Meh, none of these."

Agent: (sentiment classifier flags `frustrated` ; strategy switches)

"Got it — let me cast a wider net. Are you in the mood for something more chilled, like theatre, comedy, or a family event? Or somewhere outside Riyadh?"

Payment-Callback Flow

- 1 Agent calls Moyasar SDK → creates payment session → returns hosted-checkout URL + session ID.
- 2 Session ID stored on the booking record.
- 3 User completes payment on Moyasar's hosted page.
- 4 Moyasar fires a **webhook** to `/payments/webhook` (server-to-server).
- 5 Backend verifies the **signature** (Moyasar's webhook secret).
- 6 Backend checks the **idempotency key** — if already processed, return 200 immediately.
- 7 Booking marked `paid` ; hold converted to confirmed booking; ticket PDF generated; email sent.
- 8 Chat surfaces the result either via polling (`get_booking_status`) or via a server-pushed message.

Ticket Fraud Prevention

QR code encodes a **server-signed HMAC token** including booking ID, seat IDs, and a nonce — not just the booking ID. Server-side verification on scan. Screenshots therefore don't grant entry to anyone except the legitimate holder of a verifiable token.

Stretch Goals

LangGraph multi-agent (Catalog / Pricing / Hold / Payment / Fulfilment / Recommender); voice booking via Whisper; WhatsApp Business API channel; group-seating adjacency optimiser; refund / cancellation flow; waitlist for sold-out events; recommendation agent ("what should I see this weekend?"); Arabic-first prompts; calendar invite (.ics) attachment.

5. Data & Model

Data Sources

- **Operational data (long-term, structured).** A booking database we will design: events, venues, seats, seat-layouts, members, bookings, booking-items, holds, payments, tickets, and audit log. Seeded with a multi-event catalog (concert, sports, conference) for the demo.
- **Event assets.** Event poster images and detail URLs stored in `events.image_url` and `events.detail_url` ; rendered inline as chat cards.
- **Seat-map source.** A venue layout (rows × seats per row) generated programmatically; rendered to PNG on demand showing available / sold / held / selected states.
- **Session memory.** In-memory (or Redis) cache of conversational state.
- **Knowledge base (RAG, optional).** Venue FAQs, accessibility info, refund policy, transport directions — surfaced via retrieval when users ask.

Database Schema (suggested)

Table	Purpose
events	Event profile: title, date/time, venue, image_url, detail_url, status
venues	Venue metadata: name, city, total capacity, layout reference
seat_layouts	Per-venue grid definition (sections, rows, seat IDs)
seats	Per-event seat instance: event_id, seat_id, category, base_price, status
members	email (lookup key), tier (Bronze/Silver/Gold/Platinum), discount_pct, ticket_cap
bookings	buyer_email, event_id, status (held/pending_payment/paid/cancelled/expired), totals
booking_items	booking_id, seat_id, unit_price, discount_applied, vat
holds	booking_id, seat_id, expires_at — atomic seat locks
payments	booking_id, gateway, session_id, status, idempotency_key, signature_verified
tickets	booking_id, pdf_url, qr_token (HMAC), issued_at, emailed_at
audit_log	append-only record of state transitions for debugging and grading

Table	Purpose
<code>user_preferences</code>	email (lookup), preferred_categories, preferred_venues, price_band, rejected_categories, language, last_updated
<code>interaction_log</code>	session_id, turn_no, user_msg, agent_msg, intent, sentiment (engaged/neutral/frustrated), tools_called, timestamp

Membership-Aware Ticket Cap (proposed defaults)

Tier	Discount	Max tickets per booking
(Non-member)	0%	4
Bronze	5%	4
Silver	10%	6
Gold	12%	6
Platinum	15%	8

The "gap" between non-member (4) and Platinum (8) is the membership perk — surfaced explicitly by the agent.

Models (planned)

- **Reasoning / tool-use LLM.** A general-purpose chat model with function-calling support (candidates: OpenAI GPT-4o, Claude Sonnet, Gemini). Final pick will depend on cost, latency, and tool-calling reliability. Handles intent detection, entity extraction, tool selection, clarification, bilingual code-switching.
- **Embedding model.** OpenAI `text-embedding-3-small` or an open equivalent — only if RAG (venue FAQs / event descriptions) is enabled.
- **Speech-to-text (stretch).** Whisper for voice booking.

Evaluation Plan

- **End-to-end booking success rate** on scripted scenarios (single-ticket, multi-ticket, member, non-member, sold-out, payment-fail, hold-expiry).
- **Tool-selection correctness** — does the agent pick the right tool for the intent?
- **HITL behaviour** — does the agent always pause for approval before issuing a payment URL?
- **Concurrency safety** — under simulated concurrent requests, two users cannot hold the same seat.
- **Webhook robustness** — duplicate webhook deliveries result in exactly one ticket.
- **Discount correctness** — quote math is correct across all tiers and quantities.
- **Cap enforcement** — agent refuses (politely) when quantity exceeds the tier cap.
- **Bilingual UX** — Arabic-English code-switched inputs are parsed correctly on a hand-labelled set.
- **Personalisation lift** — preference-weighted suggestions improve the relevance score (and ultimately booking rate) vs a flat catalog on a held-out scenario set.
- **Sentiment-detection accuracy** — agent correctly classifies a hand-labelled set of user turns as engaged / neutral / frustrated.

- **Adaptive response** — when frustration is detected, the agent actually changes strategy (different category, more open question, or alternative event type), measured on a scripted frustration-recovery set.
-

6. Tech Stack

Core

- **Language:** Python 3.11+
- **Backend:** FastAPI
- **Agent framework:** LangChain (MVP) → LangGraph (multi-agent stretch)
- **ORM:** SQLAlchemy
- **Database:** SQLite (dev) / PostgreSQL (demo)
- **Chat UI:** Streamlit or Gradio (no React in MVP) — must support inline image rendering for event cards, seat maps, and ticket cards

Booking-Specific

- **Payment gateway: Moyasar sandbox** — Saudi-native, Mada / Apple Pay / STC Pay support. Team has an existing Moyasar account.
- **Seat-map renderer:** Pillow (PIL) or SVG → PNG via cairosvg; regenerated per request reflecting current availability.
- **QR codes:** `qrcode` library; payload = HMAC-signed token, not raw booking ID.
- **PDF tickets:** ReportLab or WeasyPrint; bilingual layout.
- **Email delivery:** SMTP via Gmail/SES, or SendGrid/Resend sandbox.
- **Hold TTLs:** Redis (preferred) or DB-backed with a background sweeper.
- **Webhook signing:** Moyasar's webhook secret; HMAC verification on the inbound webhook handler.

Optional / Stretch

- **Vector store:** Chroma or FAISS (venue FAQ RAG)
- **Voice:** Whisper for speech-to-text
- **WhatsApp:** WhatsApp Business API adapter
- **Calendar:** `.ics` attachment generator
- **Recommender:** event-recommendation tool ("this weekend in Riyadh")

LLM Providers

OpenAI, Anthropic, or Gemini for the reasoning model; Ollama as a local fallback.

Dev & Ops

- **Source control:** Git + GitHub
- **Spec process:** spec-kit for spec → plan → tasks workflow

- **Testing:** pytest for tools, DB integrity, concurrency (seat-hold race tests), webhook idempotency, quote math
 - **Docs:** Markdown in `/docs`
-

7. Team Introduction

Capstone team for the Agentic AI Bootcamp. The project will be built collaboratively across the four members below.

Name	Role	Contact
Nawaf Almufarej	Project lead — architecture, agent orchestration, backend	nalmufarej@gmail.com
Dana	Collaborator	—
Hessa	Collaborator	—
Refal	Collaborator	—

Final role allocation across **Agent & Tools**, **Data & DB / Payments**, **UI & Chat Rendering**, and **Eval & Demo** will be agreed at project kickoff.

Appendix — Success Criteria

The project will be considered successful when the delivered MVP demonstrates:

- A user can book one or more tickets end-to-end through chat in under 2 minutes.
- The agent never issues a payment URL without explicit user confirmation.
- Member discounts are detected by email lookup and applied correctly across all tiers.
- The ticket cap differs correctly between non-members (4) and members (up to 8), and the agent refuses cleanly when the cap is exceeded.
- Selected seats are atomically held with a visible 10-minute timer; concurrent requests for the same seat are correctly serialised.
- The Moyasar sandbox payment flow completes successfully and the webhook is processed exactly once (verified-signature + idempotency).
- After payment, a PDF ticket with an HMAC-signed QR code is generated, surfaced as a download in chat, and emailed to the buyer.
- **Returning users see preference-weighted suggestions** drawn from their past bookings and stated interests — not a generic catalog.
- **The agent detects user frustration** in a scripted "frustration-recovery" scenario and visibly changes strategy (different category, more open question, alternative event type) rather than repeating the same prompts.
- The full demo runs locally from a clean clone with one command, against the Moyasar sandbox.